Name: Abdulaziz Mohammed Alkhlawi
ID: 2240006095

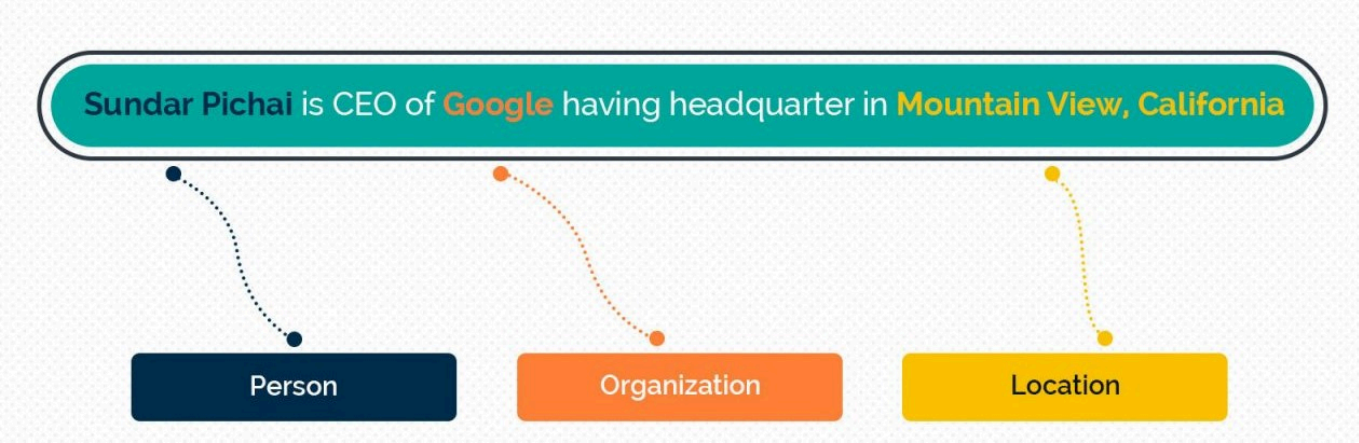
Name Entity Recognition (NER)

Completed lab solution. Original instructions: NLP_Lab9.

Run the cells from top to bottom. Results below were generated by executing this notebook. The `dataset` folder contains the original data needed by this lab. Missing Python packages are installed by the setup cell; pretrained model downloads need internet on their first run. Random seeds are fixed where supported.

```
# Install only missing dependencies into the active Jupyter Python environment.
import importlib.util, subprocess, sys, os, random
from pathlib import Path
packages = {'numpy': 'numpy', 'pandas': 'pandas', 'matplotlib': 'matplotlib', 'sklearn': 'scikit-learn',
'nltk': 'nltk', 'spacy': 'spacy'}
missing = [package for module, package in packages.items()
           if importlib.util.find_spec(module) is None]
if missing:
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', *missing])
os.environ['TOKENIZERS_PARALLELISM'] = 'false'
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
os.environ['USE_TF'] = '0'  # Transformers examples use PyTorch.
os.environ['USE_FLAX'] = '0'
import numpy as np
import pandas as pd
from IPython.display import display, Markdown, HTML
random.seed(42)
np.random.seed(42)
pd.set_option('display.max_colwidth', 90)
pd.set_option('display.max_columns', 10)
DATA = Path('dataset')
assert DATA.exists() or 9 in (7, 8, 9), 'Open this notebook from its Lab folder.'
```

Name Entity Recognition (NER) is a fundamental task in NLP that focuses on identifying and classifying entities such as names of people, organizations, locations, dates, and more within a given text. NER plays a crucial role in various NLP applications, including information retrieval, question answering, and sentiment analysis.



At its core, it is just a two-step process:

1. Detecting the entities from the text
2. Classifying them into different categories

NER using spaCy

SpaCy offers robust support for Named Entity Recognition (NER). It can recognize a variety of named entities, such as persons, organizations, dates, locations, and more. Here's more information about using spaCy for NER:



TYPE	DESCRIPTION
PERSON	People, including fictional.
NORP	Nationalities or religious or political groups.
FAC	Buildings, airports, highways, bridge, etc.
ORG	Companies, agencies, institutions etc.
GPE	Countries, cities, states.
LOC	Non-GPE locations, mountain ranges, bodies of water.
PRODUCT	Objects, vehicles, foods, etc. (Not services)
EVENT	Named hurricanes, battles, wars, sports events, etc.
WORK_OF_ART	Titles of books, songs, etc.
LAW	Named documents made into laws.
LANGUAGE	Any named language.
DATE	Absolute or relative dates or periods.
TIME	Times smaller than a day.
PERCENT	Percentage, including "%".
MONEY	Monetary values, including unit.
QUANTITY	Measurements, as of weight or distance.
ORDINAL	"first" , "second" , etc.
CARDINAL	Nuerals that do not fall under another type.

When performing Named Entity Recognition (NER) using spaCy or other NLP tools, you'll encounter various entity types, and they are represented as labels. Here are some common entity types and their labels:

Entity Type	Label	Description	Examples
Person	`PERSON`	Represents names of people.	*John Smith*, *Barack Obama*
Organization	`ORG`	Names of companies, institutions, agencies, or other organized entities.	*Microsoft*, *United Nations*
Location	`GPE`, `LOC`	Geographical locations such as cities, countries, or landmarks.	*New York*, *Eiffel Tower*
Date	`DATE`	Calendar dates, time intervals, or relative dates.	*January 1, 2023*, *tomorrow*
Time	`TIME`	Clock times or time intervals.	*2:30 PM*, *5 minutes*

Entity Type	Label	Description	Examples
Money	`MONEY`	Monetary values, including currencies and amounts.	*$100*, *€50*
Percent	`PERCENT`	Percentage values.	*50%*, *10 percent*
Product	`PRODUCT`	Names of products or goods.	*iPhone*, *Coca-Cola*
Event	`EVENT`	Names of events or occurrences.	*World Cup*, *New Year's Eve*
Law	`LAW`	Legal references or statutes.	*Copyright Act*, *First Amendment*
Language	`LANGUAGE`	Names of languages.	*English*, *Spanish*
Nationality/Religion/Political Group	`NORP`	Nationalities, religious groups, or political affiliations.	*American*, *Buddhist*

In this example, we will use spaCy to perform NER on some sample text. Make sure you have spaCy installed; if not, you can install it using : `pip install spacy`

```
import spacy
if not spacy.util.is_package('en_core_web_sm'):
    subprocess.check_call([sys.executable, '-m', 'spacy', 'download', 'en_core_web_sm'])
nlp = spacy.load('en_core_web_sm')

text = "Apple Inc. was founded by Steve Jobs in Cupertino, California. It is known for its iPhone and Mac
products."
doc = nlp(text)

print(f"Entities recognized in text : {doc.ents}") #Return a tuple of entities recognized in text

# Extract and print named entities
for entity in doc.ents:
    print(f"Entity: {entity.text}, Label: {entity.label_}")
```

```
Entities recognized in text : (Apple Inc., Steve Jobs, Cupertino, California, Mac)
Entity: Apple Inc., Label: ORG
Entity: Steve Jobs, Label: PERSON
Entity: Cupertino, Label: GPE
Entity: California, Label: GPE
Entity: Mac, Label: ORG
```

```
# Iterate through recognized entities and print their details
for ent in doc.ents:
    print(f"Entity: {ent.text}")
    print(f"Label: {ent.label_}")
    print(f"Start Position: {ent.start}")
    print(f"End Position: {ent.end}")
    print(f"Entity Type: {spacy.explain(ent.label_)}")
    print("")
```

```
Entity: Apple Inc.
Label: ORG
Start Position: 0
End Position: 2
Entity Type: Companies, agencies, institutions, etc.

Entity: Steve Jobs
Label: PERSON
Start Position: 5
End Position: 7
Entity Type: People, including fictional

Entity: Cupertino
Label: GPE
Start Position: 8
End Position: 9
Entity Type: Countries, cities, states
```

```
Entity: California
Label: GPE
Start Position: 10
End Position: 11
Entity Type: Countries, cities, states

Entity: Mac
Label: ORG
Start Position: 19
End Position: 20
Entity Type: Companies, agencies, institutions, etc.
```

Create a custom Named Entity Model

In spaCy, you can add custom entities that have not been identified in the pre-trained models by training your own Named Entity Recognition (NER) model. This involves providing labeled examples of your custom entities and then fine-tuning the model.

Here is an example of how to train a custom NER model in spaCy: https://spacy.io/usage/training#ner

```
from spacy.training.iob_utils import offsets_to_biluo_tags

# The text to annotate
text = "Apple Inc. was founded by Steve Jobs in Cupertino, California. It is known for its iPhone and Mac
products."

# --- Add custom entities annotations manually ---
annotations = [
    (text.find("Apple Inc."), text.find("Apple Inc.") + len("Apple Inc."), "ORG"),
    (text.find("Steve Jobs"), text.find("Steve Jobs") + len("Steve Jobs"), "PERSON"),
    (text.find("Cupertino, California"), text.find("Cupertino, California") + len("Cupertino, California"),
"GPE")]

# --- Step 2: Check entity alignment ---
doc = nlp.make_doc(text) #splitted to tokens
tags = offsets_to_biluo_tags(doc, annotations) #converts annotations into BILUO tags (Begin, Inside, Last,
Unit, Outside).
print("Alignment Tags:", tags[:10])
```

```
Alignment Tags: ['B-ORG', 'L-ORG', 'O', 'O', 'O', 'B-PERSON', 'L-PERSON', 'O', 'B-GPE', 'I-GPE']
```

Visualization

```
from spacy import displacy
displacy.render(nlp(text),style='ent',jupyter=True)
```

Apple Inc. **ORG** was founded by Steve Jobs **PERSON** in Cupertino **GPE** , California **GPE** . It is known for its iPhone and Mac **ORG** products.

Task 1

1. Ask a user to enter a sentence, and identify the named entities in the sentence.
2. Print the details of each entity in the sentence provided by the user.

```
# A preset answer makes Run All reproducible; set ASK_FOR_INPUT=True to enter another sentence.
ASK_FOR_INPUT=False
sentence=input('Enter a sentence: ') if ASK_FOR_INPUT else 'Abdulaziz visited Microsoft in London on Monday.'
```

```
print('Sentence:',sentence)
doc_input=nlp(sentence)
for ent in doc_input.ents:
    print(ent.text,'|',ent.label_,'|',spacy.explain(ent.label_),
          '| token span:',(ent.start,ent.end),'| character span:',(ent.start_char,ent.end_char))
if not doc_input.ents:
    print('No entities recognized.')
```

```
Sentence: Abdulaziz visited Microsoft in London on Monday.
Abdulaziz | GPE | Countries, cities, states | token span: (0, 1) | character span: (0, 9)
Microsoft | ORG | Companies, agencies, institutions, etc. | token span: (2, 3) | character span: (18, 27)
London | GPE | Countries, cities, states | token span: (4, 5) | character span: (31, 37)
Monday | DATE | Absolute or relative dates or periods | token span: (6, 7) | character span: (41, 47)
```

Task 2

Use the paragraph below to answer the following questions:

“This agreement is made and entered into by and between ‘Abc & Co.’ and ‘Bcd LLC’ for term 1 year starting from April 1, 2020, hereinafter collectively referred to as the Parties. Samuel P. Jackson in the place (New York) and on the date written below, with the following terms and conditions.”

1. Find any entity that has not been identified and add it.
2. Find the count of ‘PERSON’ and ‘ORG’ in the paragraph above.
3. Visualize the entities in the paragraph.

```
from spacy.tokens import Span
from spacy.util import filter_spans
from collections import Counter
paragraph = ('This agreement is made and entered into by and between ‘Abc & Co.’  and ‘Bcd LLC’ '
 'for term 1 year starting from April 1, 2020,  hereinafter collectively referred to as the Parties. '
 'Samuel P. Jackson in the place (New York) and on the date written below, with the following terms and
conditions.')
contract_doc=nlp(paragraph)
print('Original entities:',[(e.text,e.label_) for e in contract_doc.ents])
required={'Abc & Co.':'ORG','Bcd LLC':'ORG','Samuel P. Jackson':'PERSON',
          'New York':'GPE','April 1, 2020':'DATE','1 year':'DATE'}
manual=[]
for phrase,label in required.items():
    start=paragraph.index(phrase)
    span=contract_doc.char_span(start,start+len(phrase),label=label,alignment_mode='strict')
    assert span is not None,phrase
    manual.append(span)
# Explicit corrections replace any overlapping predictions.
remaining=[e for e in contract_doc.ents if not any(e.start<m.end and m.start<e.end for m in manual)]
contract_doc.ents=sorted(remaining+manual,key=lambda e:e.start)
print('Corrected entities:',[(e.text,e.label_) for e in contract_doc.ents])
counts=Counter(e.label_ for e in contract_doc.ents)
print('PERSON count:',counts['PERSON'])
print('ORG count:',counts['ORG'])
displacy.render(contract_doc,style='ent',jupyter=True)
```

```
Original entities:


 [('between ‘Abc & Co.', 'ORG'), ('1 year', 'DATE'), ('April 1, 2020', 'DATE'), ('Samuel P. Jackson', 'PERSON'),
('New York', 'GPE')]
Corrected entities: [('Abc & Co.', 'ORG'), ('Bcd LLC', 'ORG'), ('1 year', 'DATE'), ('April 1, 2020', 'DATE'),
('Samuel P. Jackson', 'PERSON'), ('New York', 'GPE')]
PERSON count: 1
ORG count: 2
```

This agreement is made and entered into by and between ' Abc & Co. ORG ' and ' Bcd LLC ORG ' for term 1 year DATE starting from April 1, 2020 DATE , hereinafter collectively referred to as the Parties. Samuel P. Jackson PERSON in the place (New York GPE) and on the date written below, with the following terms and conditions.

Task 3: Training a Custom NER Model

text = "Microsoft was founded by Bill Gates and Paul Allen in Albuquerque, New Mexico. It is known for its Windows operating system and Office software products."

1. Create a training example using `Example.from_dict()` with the provided text and annotations.
2. Train the NER model for **20 iterations**
3. Save the trained model in a folder named `custom_ner_model`.
4. Load the saved model and use it to identify named entities in a sentence.

```
from spacy.training import Example
from spacy.training.iob_utils import offsets_to_biluo_tags
spacy.util.fix_random_seed(42)
training_text = ('Microsoft was founded by Bill Gates and Paul Allen in Albuquerque, New Mexico. '
 'It is known for its Windows operating system and Office software products.')
entity_labels={'Microsoft':'ORG','Bill Gates':'PERSON','Paul Allen':'PERSON',
               'Albuquerque':'GPE','New Mexico':'GPE','Windows':'PRODUCT','Office':'PRODUCT'}
entities=[(training_text.index(phrase),training_text.index(phrase)+len(phrase),label)
          for phrase,label in entity_labels.items()]
custom_nlp=spacy.blank('en')
ner=custom_nlp.add_pipe('ner')
for label in set(entity_labels.values()): ner.add_label(label)
annotation={'entities':entities}
assert '-' not in offsets_to_biluo_tags(custom_nlp.make_doc(training_text),entities)
example=Example.from_dict(custom_nlp.make_doc(training_text),annotation)
optimizer=custom_nlp.initialize(get_examples=lambda:[example])
for iteration in range(20):
    example=Example.from_dict(custom_nlp.make_doc(training_text),annotation)
    losses={}
    custom_nlp.update([example],sgd=optimizer,drop=0.0,losses=losses)
    print(f'Iteration {iteration+1:2d}/20 | NER loss: {losses["ner"]:.4f}')
model_path=Path('custom_ner_model')
custom_nlp.to_disk(model_path)
loaded_nlp=spacy.load(model_path)
print('Saved and reloaded:',model_path)
print('Training sentence entities:',[(e.text,e.label_) for e in loaded_nlp(training_text).ents])
test_sentence='Bill Gates founded Microsoft.'
print('New sentence:',test_sentence)
print('Predicted entities:',[(e.text,e.label_) for e in loaded_nlp(test_sentence).ents])
displacy.render(loaded_nlp(training_text),style='ent',jupyter=True)
```

```
Iteration  1/20 | NER loss: 16.4444
Iteration  2/20 | NER loss: 26.4833
Iteration  3/20 | NER loss: 26.2558
Iteration  4/20 | NER loss: 25.8843
Iteration  5/20 | NER loss: 25.2662
Iteration  6/20 | NER loss: 24.3288
Iteration  7/20 | NER loss: 22.9318
Iteration  8/20 | NER loss: 20.6413


Iteration  9/20 | NER loss: 17.4721
Iteration 10/20 | NER loss: 13.4208
Iteration 11/20 | NER loss: 9.5739
Iteration 12/20 | NER loss: 7.6619
Iteration 13/20 | NER loss: 6.7663
Iteration 14/20 | NER loss: 5.9038
Iteration 15/20 | NER loss: 4.8513
```

```
Iteration 16/20 | NER loss: 29.7098
Iteration 17/20 | NER loss: 9.2642


Iteration 18/20 | NER loss: 8.3172
Iteration 19/20 | NER loss: 6.6880
Iteration 20/20 | NER loss: 4.9263


Saved and reloaded: custom_ner_model
Training sentence entities: [('Microsoft', 'ORG'), ('Bill Gates', 'PERSON'), ('Paul Allen', 'PERSON'),
('Albuquerque', 'GPE'), ('New Mexico', 'GPE'), ('Windows', 'PRODUCT'), ('Office', 'PRODUCT')]
New sentence: Bill Gates founded Microsoft.
Predicted entities: [('Bill Gates', 'PERSON')]
```

Microsoft ORG was founded by Bill Gates PERSON and Paul Allen PERSON in Albuquerque GPE , New Mexico GPE . It is known for its Windows PRODUCT operating system and Office PRODUCT software products.

Observations

The corrected contract contains one PERSON and two ORG entities. Manual spans correct the supplied paragraph only; assigning `doc.ents` does not train the general model. The custom model is trained for exactly 20 iterations using valid character offsets and `Example.from_dict`, then saved and reloaded from `custom_ner_model`. One annotated sentence is enough to demonstrate the training API but cannot establish generalization; even predictions on its training sentence can be incomplete after only 20 updates. No independent NER accuracy is claimed.